

`write_reg` is written as follows:

```
static int ds1347_write_reg(struct device *dev, unsigned char
address, unsigned char data)
{
    struct spi_device *spi = to_spi_device(dev);
    unsigned char buf[2];

    buf[0] = address & 0x7F;
    buf[1] = data;

    return spi_write_then_read(spi, buf, 2, NULL, 0);
}
```

The parameters to the routine are the address to which the data has to be written and the data which has to be written to the device. `spi_write_then_read` is the routine that has the following parameters:

`struct spi_device`: The slave device to be written.
`tx_buf`: Transmission buffer. This can be NULL if reception only.
`tx_no_bytes`: The number of bytes in the `tx buffer`.
`rx_buf`: Receive buffer. This can be NULL if transmission only.
`rx_no_bytes`: The number of bytes in the receive buffer.
 In the case of the RTC DS1347 write routine, only two bytes are to be written: one is the address and the other is the data on that address.

The reading of the SPI bus is done as follows:

1. First, the address of the SPI slave device is written on the SPI bus. In the case of RTC DS1347, the address should contain its most significant bit set for the read operation (as per the DS1347 datasheet).

2. Then the data is read from the SPI bus.

Since this is a common operation, a separate routine, `ds1347_read_reg`, is written as follows:

```
static int ds1347_read_reg(struct device *dev, unsigned char
address, unsigned char *data)
{
    struct spi_device *spi = to_spi_device(dev);

    *data = address | 0x80;

    return spi_write_then_read(spi, data, 1, data, 1);
}
```

In the case of RTC DS1347, only one byte, which is the address, is written on the SPI bus and one byte is to be read from the SPI device.

RTC DS1347 driver probe routine

When the probe routine is called, it passes an `spi_device` struct, which was created when `spi_board_info` was

registered. The first thing the probe routine does is to set the SPI parameters to be used to write on the bus. The parameters are the mode in which the SPI device works. In the case of RTC DS1347, it works on Mode 3 of the SPI:

```
spi->mode = SPI_MODE_3;
```

`bits_per_word` is the number of bits transferred. In the case of RTC DS1347, it is 8 bits.

```
spi->bits_per_word = 8;
```

After changing the parameters, the kernel has to be informed of the changes, which is done by calling the `spi_setup` routine as follows:

```
spi_setup(spi);
```

The following steps are carried out to check and configure the RTC DS1347.

1. First, the RTC control register is read to see if the RTC is present and if it responds to the read command.
2. Then the write protection of the RTC is disabled so that the code is able to write on the RTC registers.
3. Then the oscillator of the RTC DS1347 is started so that the RTC starts working.

Till this point the kernel is informed that the RTC is on an SPI bus and it is configured. After the RTC is ready to be read and written by the user, the read and write routines of the RTC are to be registered with the Linux kernel RTC subsystem as follows:

```
rtc = devm_rtc_device_register(&spi->dev, "ds1347", &ds1347_
rtc_ops, THIS_MODULE);
```

The parameters are the name of the RTC driver, the RTC operation structure that contains the read and write operations of the RTC, and the owner of the module. After this registration, the Linux kernel will be able to read and write on the RTC of the system. The RTC operation structure is filled as follows:

```
static const struct rtc_class_ops ds1347_rtc_ops = {
    .read_time = ds1347_read_time,
    .set_time = ds1347_set_time,
};
```

The RTC read routine is implemented as follows.

The RTC read routine has two parameters, one is the device object and the other is the pointer to the Linux RTC time structure struct, `rtc_time`.

The `rtc_time` structure has the following fields, which have to be filled by the driver:

`tm_sec`: seconds (0 to 59, same as RTC DS1347)

`tm_min`: minutes (0 to 59, same as RTC DS1347)